Choosing Among Correct Proofs: What LEVER Optimizes While the Lean Kernel Checks

Mathematicians care which correct proof they obtain because valid proofs of a theorem differ in simplicity, purity, and search compute (Section 1). Most LLM provers search for any verified proof and improve it only afterwards through post-hoc refactoring, which revisits a proof already paid for and keeps an argument that was chosen for a different objective. On 8 October 2026, Nihal Jain, Shuangjie Yao, Begum Cicekdag, Zhuo Zhang, and Suman Jana at Columbia University submitted "LEVER: Adaptive Cost-Aware Proof Search Over AND/OR Graphs" (`arXiv:2610.11862v1`). LEVER makes the objective over correct proofs programmable and optimizes it during search while the Lean 4 kernel checks every decomposition and final proof under standard axioms without `sorry`, `admit`, `axiom`, or `native_decide` (Section 4.2).

AND/OR proof graphs and scoring partial proofs

LEVER models partial proofs as a bipartite directed acyclic AND/OR graph alternating between OR and AND nodes (Section 3.1). Goals (theorems or lemmas) are OR nodes, with root r as the main theorem, and decompositions into subgoals are AND nodes checked by the Lean kernel. An OR node is solved if any child is solved, and an AND node if all children are solved. Childless AND nodes are proved directly. A solution subgraph contains r, one AND child per OR node, and all OR children per AND node.

With nonnegative edge costs `c(n, d)`, decompositions pay for all subgoals whereas goals take their cheapest proof. Values back up by summing at AND nodes, `V(n) = (c(n, d) + V(d))`, and taking the minimum at OR nodes, `V(n) = min (c(n, d) + V(d))`, with `V = 0` at solved leaves and `V =` at false goals. LEVER scores each partial proof as realized cost on proved parts plus predicted value on open subgoals, expanding the best-scoring partial proof first.

Priors, redraws, and the programmable objective

Classical AO* assumes deterministic expansion. Because LLM expansion at temperature 1.0 is stochastic, drawing from an expanded OR node again can return a cheaper decomposition (Section 3.3). Every OR node g therefore carries an unexpanded prior AND child holding an estimate `V(g)` of finishing g via a fresh draw. Each iteration of Algorithm 1 runs three steps:

- **Selection.** Follow the minimizing child at each OR node and all children at each AND node from r. When an OR node's minimizing child is its prior, expand that goal (up to 25 concurrently, Appendix A). Re-expanding a goal with existing children is a redraw, which must beat the incumbent by factor `( = exp()` on log scales, equal to 1.85 for compute cost, 1.50 for length, and 1.72 for impurity, Table 4).
- **Expansion.** Each expansion is one LLM agent conversation with a Lean workspace that has tools for Lean compilation (Kimina Lean Server), Mathlib retrieval, and Python (Section 4.2). The agent writes a Lean proof that relies on open subgoals, proving small helpers in place and annotating dependencies among open subgoals with LeanArchitect's `sorry_using` tactic. The goal-to-decomposition edge takes the realized cost, new subgoals get priors, and decomposition-to-subgoal edges cost zero.
- **Backpropagation and posterior update.** Costs pass upward through the AND/OR backup equations and update priors along the path. On a log scale, fresh draws of

> g are normal around mean _g with spread , and initial prediction V_0(g) has error . After k draws with costs v_1, ..., v_k, the posterior mean (Equation 1) is _g = (n log V_0(g) + log v_i) / (n + k) with n = š / š, giving V(g) = exp(_g). The prior counts as n draws (0.33 for compute cost, 0.32 for length, and 0.05 linearly for impurity, Table 4) and each realized draw as one. Failed attempts add one .

Because past spend is sunk, LEVER optimizes only remaining compute cost comp (V_comp), where realized edges add zero and open goals add their prior (Section 3.3). Combining V_comp with additive metrics V_m weighted by _m dollars per unit yields C(n) = V_comp(n) + _m _m V_m(n). For comp and proof length (len), DeepSeek-V4-Flash with 40 calibration examples assigns open goals to seven $\log_2$ bins ($0.016 to $0.512 for comp, and 512 to 16,384 Lean tokens for len, scaled by 0.2 to favor shorter proofs, Section 4.4). Value calls (0.89% of spend) and one planner call (capped at $0.20, Appendix A) are included in all reported costs. For topical impurity (imp), predicting 0 caused unstable redraws because open goals looked pure, so open goals start at 4 references (the 25th calibration percentile).

Topical impurity and the unit triangle example

Following work on purity of method (Detlefsen and Arana, 2011, Martinot, 2024), LEVER reads topical impurity off Mathlib's structure via #lever_uses (Section 4.3, Appendix B). Every Mathlib name has a namespace (such as ZMod in ZMod.wilsons_lemma) and a three-level directory (such as Mathlib/NumberTheory). A theorem's subject is the set of namespaces and directories of the names its statement uses. A referenced theorem is impure if it falls outside the subject on both accounts, while a root-namespace lemma such as mul_pos is impure only if it also lies outside a 12-directory infrastructure set (such as Mathlib/Data/Nat). In Figure 3, std_tri_volume states that the unit triangle has area 1/2 using MeasureTheory.volume and convexHull. Plan 1 slices the triangle and integrates via intervalIntegral.integral_sub, flagging 25 impure references because the statement mentions no integrals. Plan 2 halves the unit square in measure theory via MeasureTheory.measure_union_ad flagging 6 references.

PutnamBench results on compute cost and proof quality

Experiments use PutnamBench (672 Lean 4 v4.31.0 problems) with DeepSeek-V4-Flash-0731 for all calls at $0.22, $0.007, and $0.66 per million input, cached input, and output tokens (Section 4.1). Taking ranks 51 through 150 of the NearAI (2026) single-conversation run (20 calibration and 80 evaluation problems, Appendix C.1), the authors re-price main-text costs at a fixed 90% cache-hit rate (between NearAI's 93.7% per-call rate and LEVER's 86.2%).

Under a $3 budget on the 80 evaluation problems (Section 5.1, Table 1, charging unsolved runs $3 and proving goals five levels deep or under bin 5 directly), LEVER solves 96.3% of problems versus 80.0% for NearAI, at a mean cost of $0.95 versus $1.44 (34% less). Local subgoal contexts cut input tokens per problem from 42.0M to 17.9M (0.62M versus 0.36M output tokens). At actual cache rates (Appendix C.3, Table 6), LEVER still solves 96.3% versus 86.3% at $0.90 versus $1.07 per problem (16% cheaper). On putnam_1972_a3 (Appendix C.4.1), LEVER rejects a three-subgoal decomposition priced at $1.54 in total against a $0.72 root prior before attempting any subgoal (the margin is = 1.85). It then draws a second decomposition whose two subgoals are priced at $0.91 in total, proves both, and finishes the run for $1.75.

On $N = 20$ solved problems under a $4.5 budget (Section 5.2, Figure 5, Table 2), LEVER is compared against NearAI with prompt steering (`+Instruct`) and post-hoc rewriting (`+Refactor`, charged initial plus refactoring cost):

- **Topical impurity.** At `= 0.1`, LEVER cuts impurity by 42% at $1.85 per problem (-15% at `= 0.01`, +1% at `= 0`), beating `+Refactor` (-33% at $2.88) at two-thirds of the cost with lower per-problem standard deviation (32 versus 101 points, with `+Refactor` rising from 29 to 132 impure references on `putnam_2007_a2`). `+Instruct` was 11% less pure than the baseline.
- **Proof length and cross-metric effects.** At `= 10ş`, LEVER cuts length by 21% at $2.59 (-5% at `= 10`, +4% at `= 0`), while `+Refactor` reaches -31% at $1.83, so refactoring still wins on length. Optimizing length (`= 10ş`) also lowered impurity by 18%, whereas optimizing impurity (`= 0.1`) made proofs 10% longer (+10%).

Limitations and implications for formalization

Several limitations apply. Topical impurity is a proxy. Mathlib's organization stands in for subject, the 12-directory infrastructure set comes from one pilot corpus, and references are counted unweighted (Section 4.3, Appendix B). Because helper statements do not charge definitions, local re-proofs could game impurity. On the 20 quality problems, LEVER at `= 0.1` wrote 36.1 helpers per proof versus 25.9 at `= 0`, and 11.2 versus 10.1 off-subject helpers (NearAI wrote 14.1), changing the 42% reduction by under one point when penalized. Length can similarly be gamed by tactics such as `nlinarith`. Evaluation uses a selected PutnamBench slice (ranks 51 through 150, not the hardest 50 which took up to 5.5 days), one model (`DeepSeek-V4-Flash-0731`), and costs re-priced at a 90% cache rate. The Lean-verified exactness proof (`lemma1_exact_values`, Appendix D) assumes exact priors and tree unfolding, leaving noisy-value analysis for future work and charging shared subgoals to one owner because exact shared-subgoal optimization is NP-hard (Sahni, 1974). Finally, `+Instruct` and `+Refactor` were built on NearAI because dedicated refactoring tools (Lean Refactor, ProofOptimizer) were unavailable or on Lean v4.9 (ImProver).

For people formalizing mathematics in Lean, LEVER separates verification from proof selection. The kernel guarantees that any accepted proof is sound, while the search objective turns properties of the proof artifact, including library dependencies, token length, and compute spend, into parameters chosen before search begins.

Contributor: Alessandro Linzi